

Real Estate Machine Learning Project

Predicting Real Estate Prices

A Training Project Report in Data Analysis and Machine Learning

Prepared by the Team (10 members)

Adam Yasser Sayed

Mohammed Alaa korani

Yehia abdellhmed Yehia

Omar Mohamed Ahmed

Mohamed Adel Elsayed

Tony Milad farouq

Mohamed Fareed Mohamed

Peter mohep kerolse Gregs

Mohamed Alaa El-Din Mahmoud

Yousef Osama Awad

Data Analyst Training | 2026

Table of Contents

1. Introduction

2. Project Overview

3. Data Used

4. Data Cleaning

5. Exploratory Data Analysis (EDA)

6. Regression Modeling: Predicting Property Price

7. Feature Engineering

8. Classification Modeling: Predicting Sale Status

9. Interactive Web App (Streamlit App)

10. Tools and Technologies Used

11. Task Distribution Among Team Members

12. Challenges and Limitations

13. Future Recommendations

14. Conclusion

15. Appendix: Project File Structure

1. Introduction

This report provides complete documentation of a training project in the field of Data Analysis and Machine Learning, carried out by a team of ten members, divided into four sub-teams: the Data & Exploratory Analysis team, the Modeling team, the Web App Development team, and the Presentation team.

The project aims to apply a complete Data Science Lifecycle to real estate data — starting from data collection and cleaning, through exploratory analysis and insight extraction, to building machine learning models that predict property price (a regression problem) and classify whether a property will be sold or not (a classification problem) — and finally wrapping the results inside an interactive web app built with Streamlit.

This report also outlines the tools and techniques used at each stage, the methodology followed, the results obtained, along with the challenges faced by the team, lessons learned, and recommendations that could improve the project's outcomes in future iterations.

1.1 Project Objectives

- Train the team to practically apply the steps of data science on a realistic dataset (properties in Greater Cairo).
- Build a regression model to predict property price (EGP_Price) based on its features.
- Build a classification model to predict property sale status (Is_Sold).
- Compare several machine learning algorithms (Linear Regression, Decision Tree, Random Forest, XGBoost, Gradient Boosting) and evaluate their performance.
- Apply hyperparameter tuning and cross-validation techniques to improve the models.
- Develop an interactive web app that lets any user input property data and get an instant price and sale-status prediction.
- Organized teamwork with a clear distribution of roles among the ten team members.

1.2 Report Scope

This report covers all project stages as reflected in the actual project file structure: the Data, Exploratory Data Analysis (EDA), Modeling (covering both regression and classification), and the Web App (Streamlit App) — in addition to a section outlining the tools used, task distribution, challenges, recommendations, and conclusion.

2. Project Overview

"Real Estate Price Predictor" is a comprehensive training project that simulates the work of a data analysis team inside a real estate company. The project relies on a generated dataset of 250 properties across five major residential districts in Greater Cairo, aiming to extract marketing insights on one hand, and build predictive models that help price properties and estimate their likelihood of being sold on the other.

The project structure is distinguished by clear organization into sequential stages, where each stage's outputs feed the next stage's inputs, starting from the raw data through to the final web app:

- Stage 1 – Data (1_Data): raw data, cleaned data, and the data dictionary.
- Stage 2 – Exploratory Data Analysis (2_EDA): data cleaning, insight extraction, and five explanatory charts.

- Stage 3 – Modeling (3_Modeling): split into regression modeling (01_Regression) for price prediction, and classification modeling (02_Classification) for sale-status prediction.
- Stage 4 – Web App (4_App_Streamlit): a five-page interactive app to display results and enable live predictions.
- Stage 5 – Presentation (5_Presentation) and Demo (6_Demo): to summarize and present the project to the evaluation committee.

2.1 Project Idea in Brief

The core idea is to answer two practical questions that any real estate company faces: (1) How much should a given property be priced, based on its features? and (2) How likely is this property to be sold in the future? To answer these two questions, the team used two complementary machine learning approaches: Regression, to predict a continuous numeric value (the price), and Classification, to predict a binary category — "sold" or "not yet sold".

3. Data Used

The first stage of the project relies on a ready-made real estate dataset provided by a real estate company, as described in the data dictionary. The dataset consists of 250 properties (rows) and 9 feature columns. The table below shows the full data dictionary:

Column	Type	Description
Property_ID	Text	Unique identifier for each property
Location	Categorical	Property location (5 districts)
Property_Type	Categorical	Property type: Apartment / Villa / Duplex / Penthouse
SqM_Area	Numeric	Property area in square meters
Bedrooms	Numeric	Number of bedrooms (1 to 5)
Bathrooms	Numeric	Number of bathrooms (1 to 3)
EGP_Price	Numeric	Property price in Egyptian Pounds (regression target)
Is_Sold	Categorical	Was the property sold? (No / Yes) — classification target
Days_On_Market	Numeric	Number of days the property was listed for sale

3.1 Available Locations and Property Types

The data covers five main residential districts: Nasr City, New Cairo, Maadi, Sheikh Zayed, and 5th Settlement. There are four property types: Apartment, Villa, Penthouse, and Duplex.

3.2 Data Size Overview

Metric	Value
Number of rows (properties)	250
Number of columns (features)	9
Missing values	0 (none)

Metric	Value
Duplicate rows	0 (none)
Data source	Real Estate Company

4. Data Cleaning

The Data & EDA team carried out the data cleaning step inside a dedicated notebook file (01_data_cleaning.ipynb), using the pandas and numpy libraries in Python. This step included the following procedures:

- Loading the raw data (raw_data.csv) and checking its shape and column names.
- Checking for missing values — confirmed there were none in any column.
- Checking for duplicate rows — confirmed there were no duplicates in the data.
- Reviewing data types for each column to ensure they matched the description in the data dictionary.
- Extracting basic descriptive statistics (mean, standard deviation, minimum, and maximum) for each numeric column.
- Saving a clean copy of the data as cleaned_data.csv, to be used in all subsequent stages (EDA and modeling).

Given the high quality of the source data (no missing values or duplicates), this stage served more as a validation step than an intensive cleaning process — paving the way directly for the exploratory analysis stage.

5. Exploratory Data Analysis (EDA)

The data team carried out the exploratory analysis stage inside the file 02_eda_analysis.ipynb, using the pandas, numpy, matplotlib, and seaborn libraries, with the aim of understanding the data distribution and the relationships between different variables, and extracting five key charts that illustrate the most important aspects of the dataset.

5.1 Key Insights

- The average property price across the entire dataset is EGP 8,470,125.
- Property prices range between a minimum of EGP 1,566,150 and a maximum of EGP 14,997,927.
- The most common property type is "Apartment," with 127 properties, representing about 50.8% of all properties.
- The least common property type is "Duplex," with only 29 properties, representing about 11.6%.
- The highest average price by district was in 5th Settlement (EGP 8,785,033), while the lowest was in Sheikh Zayed (EGP 7,899,658).
- The linear relationship between property area and price is very weak (correlation coefficient r = 0.059), suggesting that other factors — such as location and property type — may play a larger role in determining price.
- No outliers were detected using the 1.5×IQR rule on the numeric columns.

5.2 Charts

Chart 1 — Average Price by Location



Figure 1: Average property price in each of the five districts, showing a clear convergence between districts, led by 5th Settlement.

Chart 2 — Property Type Distribution

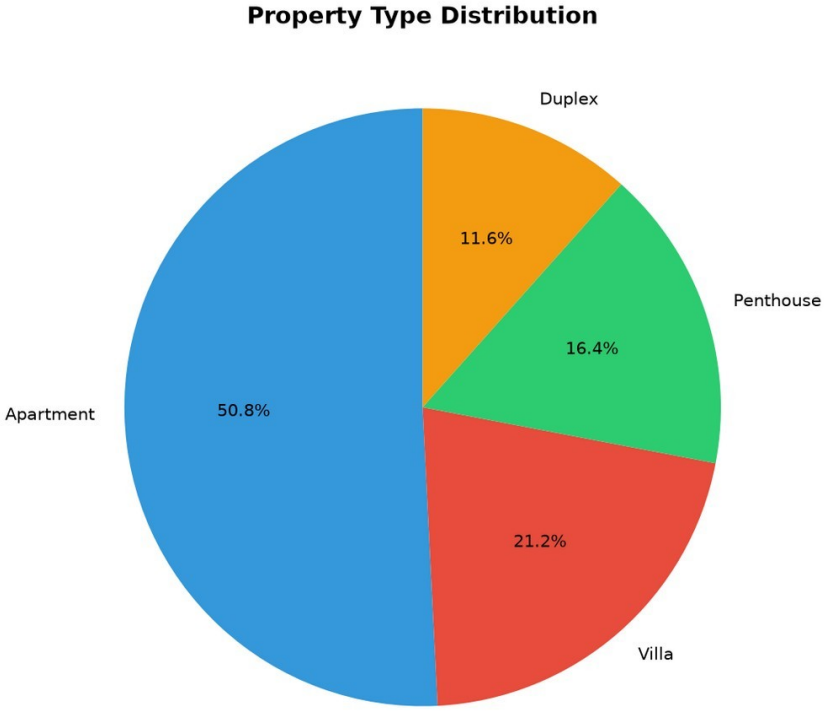


Figure 2: Relative distribution of the four property types within the dataset.

Chart 3 — Price Distribution

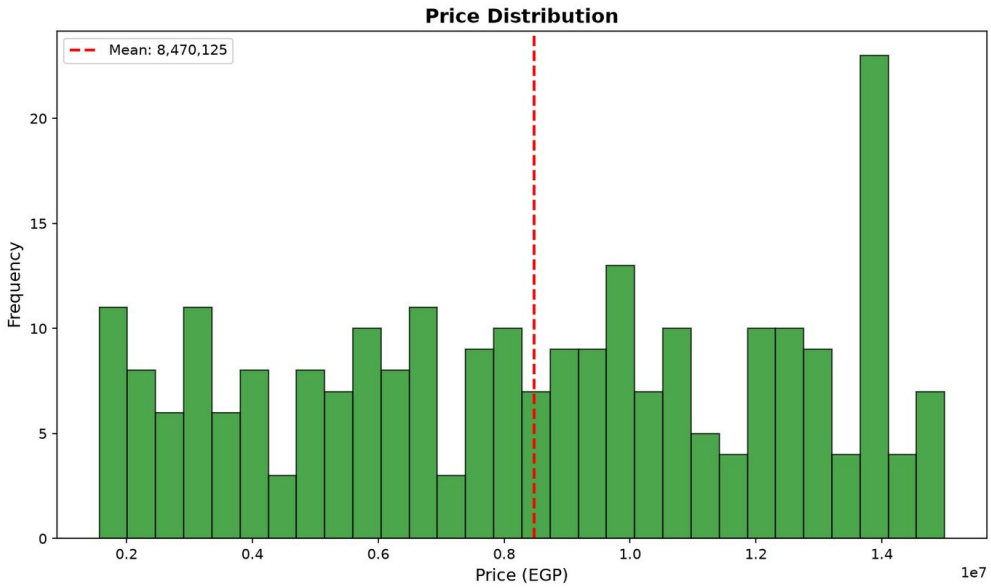


Figure 3: Histogram showing the distribution of property prices, with the mean line.

Chart 4 — Area vs. Price



Figure 4: Scatter plot between area and price, showing the absence of a clear linear relationship between them.

Chart 5 — Correlation Heatmap



Figure 5: Heatmap of correlation coefficients between all numeric variables in the data.

5.3 EDA Summary

The EDA stage concludes that the dataset is completely clean (no missing or duplicate values), but the linear relationships between the available numeric variables (area, number of bedrooms, number of bathrooms) and property price are generally weak — a pivotal conclusion that would directly affect the performance of the regression models in the next stage of the project, as becomes clear later on.

6. Regression Modeling: Predicting Property Price

The Modeling team (3 members) took on the task of building machine learning models to predict property price (EGP_Price). Work was divided among them across three sequential notebook files, with each file building on the outputs of the previous one.

6.1 Preprocessing Steps

- Dropping the Property_ID identifier column, as it holds no predictive value.
- Converting the Is_Sold column from text values (No/Yes) into numeric values (1/0).
- Encoding the categorical columns Location and Property_Type using One-Hot Encoding (via pandas.get_dummies), dropping the first category of each column to avoid multicollinearity.
- Splitting the data into training and test sets at an 80/20 ratio using train_test_split, with random_state=42 fixed to ensure reproducibility of results.
- Applying StandardScaler feature scaling only before training the linear regression model, while the tree-based models were trained on the unscaled data.

6.2 Model 1: Linear Regression and Decision Tree

Two baseline models were trained in the file 01_linear_dt_youssef.ipynb: a Linear Regression model on the scaled data, and a Decision Tree Regressor with a maximum depth of 10. The fitted scaling tool (scaler.pkl) was saved for later use in the web app.

6.3 Model 2: Random Forest and XGBoost

Two more advanced models were built in the file 02_adel_rf_xgb.ipynb: a Random Forest Regressor with 100 trees and a maximum depth of 10, and an XGBRegressor with 100 trees, a learning rate of 0.1, and a maximum depth of 5. A feature-importance chart was also extracted based on the XGBoost model.

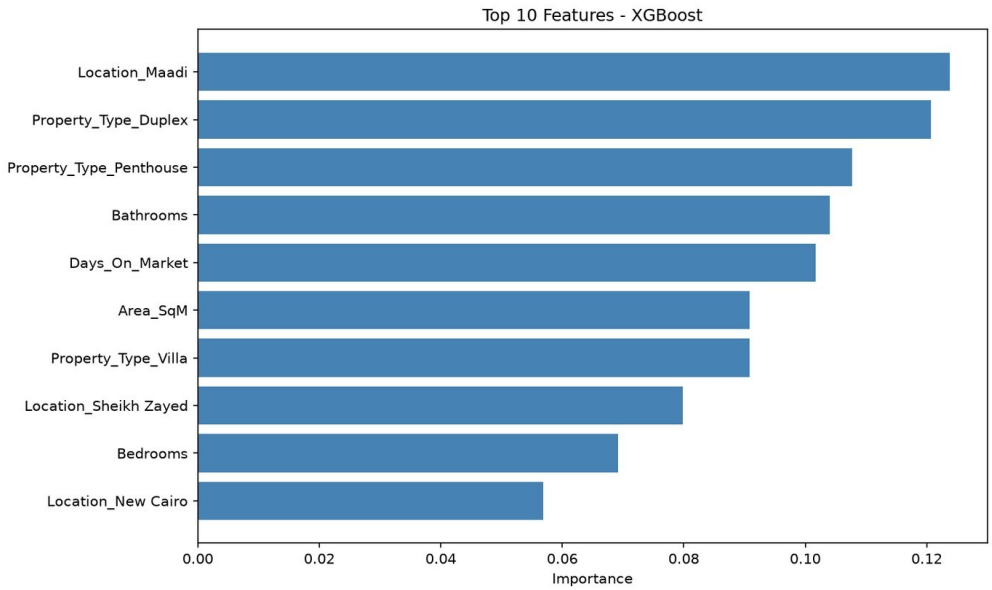


Figure 6: Top 10 features affecting the XGBoost model's price predictions, led by the Maadi location, property type (Duplex/Penthouse), and number of bathrooms.

6.4 Hyperparameter Tuning

A team member took on the task of gathering and comparing colleagues' results in the file 03_tuning_best_farid.ipynb, then performed careful hyperparameter tuning for both XGBoost and Random Forest using GridSearchCV with 5-fold cross-validation, according to the following grids:

Model	Tuned Parameters
XGBoost	n_estimators: [100, 200, 300] | learning_rate: [0.01, 0.05, 0.1] | max_depth: [3, 5, 7]
Random Forest	n_estimators: [100, 200] | max_depth: [5, 10, 15] | min_samples_split: [2, 5]

6.5 Final Results Comparison

All models were evaluated on the test set using three metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). The table below summarizes the final results, ranked in descending order of R^2:

Rank	Model	MAE	RMSE	R^2
1	Linear Regression	3,157,737	3,866,100	-0.033

Rank	Model	MAE	RMSE	R^2
2	Tuned XGBoost	3,231,462	3,886,699	-0.044
3	Tuned Random Forest	3,248,975	3,926,591	-0.066
4	Random Forest	3,309,401	4,004,579	-0.109
5	XGBoost	3,759,825	4,410,333	-0.345
6	Decision Tree	3,583,859	4,619,457	-0.475

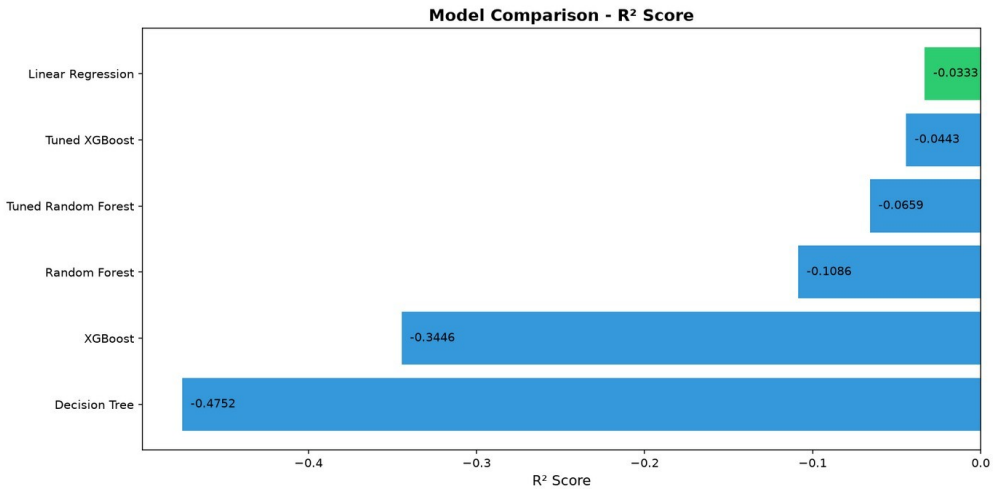


Figure 7: Visual comparison of R^2 values across the six models — Linear Regression is closest to zero (best relative performance).

6.6 A Critical Reading of the Results: Why Were the R^2 Values Negative?

One of the most important findings to present with scientific honesty is that all six regression models recorded negative R^2 values on the test data. A negative R^2 means that a model performed worse than a very simple model that always predicts the average price as a fixed value for every property. This result is not a coding error — it is a direct reflection of what was observed during the EDA stage: the relationship between the available features (area, number of bedrooms, location, type) and property price is very weak in this data (recall that the correlation between area and price did not exceed 0.059).

It is notable that the best-performing regression model is also the simplest one (Linear Regression), while the more complex models (such as Decision Tree and the untuned XGBoost) performed the worst, because they tend to overfit random patterns in the training data that do not recur in the test data — especially given the relatively small dataset (only 250 properties). Hyperparameter tuning did noticeably improve XGBoost and Random Forest compared to their untuned versions, but this was not enough to overcome the performance ceiling imposed by the nature of the data itself.

6.7 Saving the Final Model

Based on the comparison, the Linear Regression model was adopted as the best model, and was saved along with its related files for later use in the web app:

- best_model.pkl — the final trained model (Linear Regression).
- scaler.pkl — the scaling tool (StandardScaler) used before prediction.
- feature_columns.pkl — the list of feature names in the order required by the model.

- model_info.pkl — descriptive metadata about the model (name, features, target, and whether scaling is required).

7. Feature Engineering

Before moving to classification modeling, the team added seven new engineered features to provide the models with deeper information that was not directly visible in the original columns, in the file 05_feature_engineering.ipynb, as follows:

New Feature	Calculation Method
Total_Rooms	Bedrooms + Bathrooms
Area_per_Bedroom	Area_SqM ÷ Bedrooms
Ratio_Bathroom	Bedrooms ÷ Bathrooms
Is_Luxury	1 if Area_SqM > 300 and Bedrooms ≥ 4, else 0
Price_per_SqM	Price_EGP ÷ Area_SqM
Is_Expensive	1 if Price_per_SqM is above the overall median, else 0
Category_Days	Categorizes Days_On_Market into: Fast (0–30) / Medium (30–90) / Slow (90–180)

These features aim to capture non-linear, composite patterns — such as how "luxurious" a property is (Is_Luxury), or how competitive its price is relative to its area (Price_per_SqM) — which are expected to be more strongly correlated with how quickly a property sells than the raw features alone.

8. Classification Modeling: Predicting Sale Status (Is_Sold)

The Modeling team also tackled a binary classification problem: will the property be sold or not? This model was developed across three consecutive versions (V1, V2, V3), each building on the previous one and attempting to improve performance.

8.1 Version 1 (V1) — Baseline Model

In this version, only the original features were used, after One-Hot Encoding. Two models were trained: a Random Forest Classifier (100 trees, max depth 10) and an XGBoost Classifier (100 trees, max depth 5), with a Stratified Split to preserve the class ratio between training and test sets.

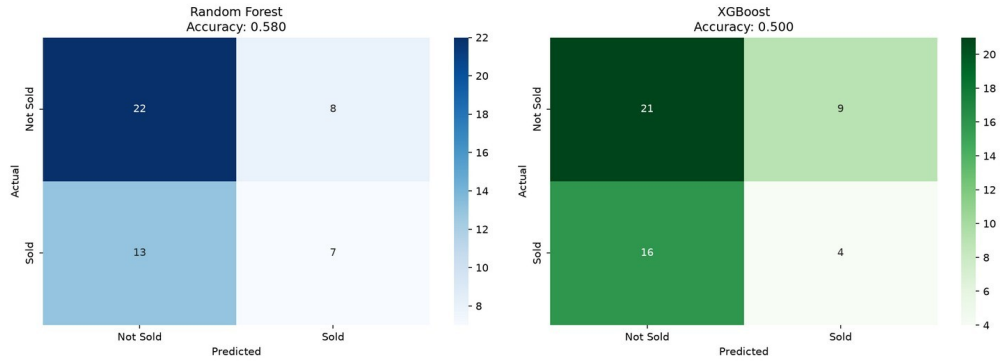


Figure 8: Confusion matrices for the two baseline models in Version 1 — Random Forest (Accuracy: 0.580) and XGBoost (Accuracy: 0.500).

8.2 Version 2 (V2) — After Feature Engineering

After adding the seven new features, three more advanced models were trained: Random Forest (300 trees, depth 15, with class_weight='balanced'), Gradient Boosting Classifier (200 estimators, learning rate 0.05), and XGBoost (300 estimators, using scale_pos_weight to address class imbalance). The three models were also evaluated via 5-fold cross-validation.

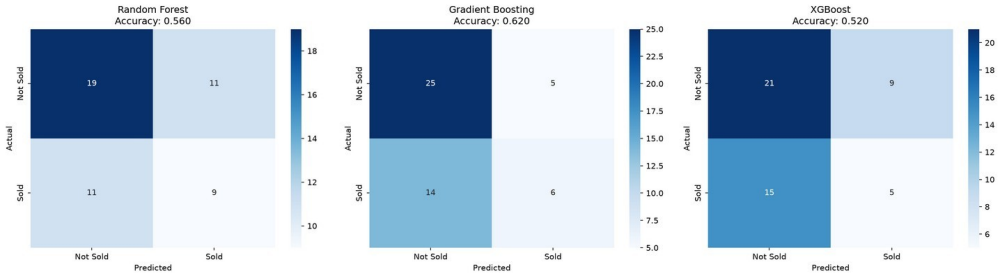


Figure 9: Confusion matrices for the three models after feature engineering in Version 2 — Random Forest (0.560), Gradient Boosting (0.620), XGBoost (0.520).

8.3 Version 3 (V3) — Final Cross-Validation

In the final version, 10-fold Stratified Cross-Validation was used to compare the three models more reliably. The best model was then selected based on average cross-validation accuracy, and retrained on a final 80/20 split to measure its actual performance on data it had not seen before.

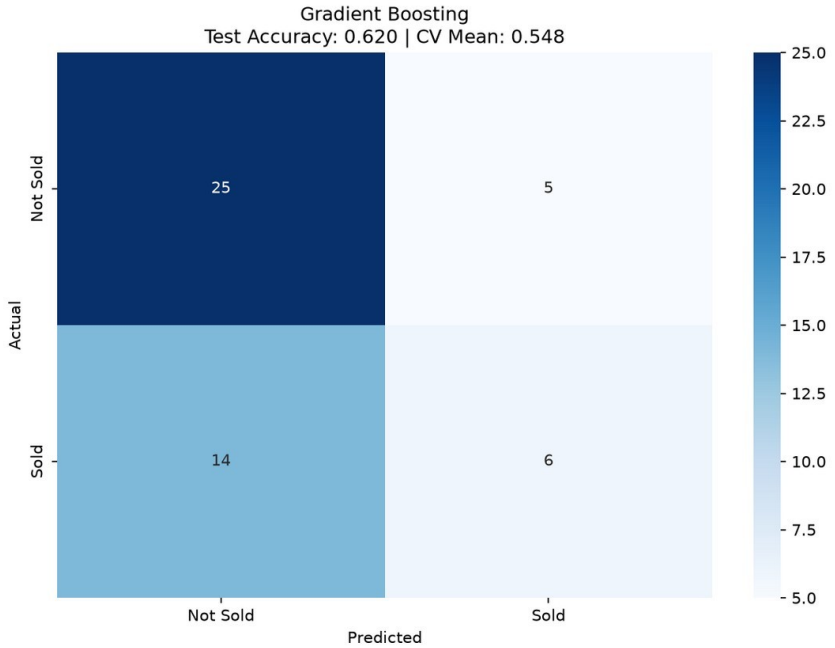


Figure 10: Confusion matrix for the final model (Gradient Boosting) — Test Accuracy 62.0%, average cross-validation accuracy 54.8%.

8.4 Final Result Analysis

The winning model in the final version was the Gradient Boosting Classifier, with a Test Accuracy of 62.0% and an average cross-validation accuracy of 54.8%. While this is higher than random guessing on a binary problem (50%), the confusion matrix reveals a clear shortfall: the model correctly identified 25 out of 30 "Not Sold" properties, but only 6 out of 20 "Sold" properties (a Recall of about 30% for the positive class).

This shortfall reflects a common challenge in classification problems with class imbalance, where the model tends to favor the more frequent class ("Not Sold"). Despite attempts to address this bias using class_weight and scale_pos_weight, the small size of the dataset (250 properties, of which only about 100 were sold) limited the models' ability to learn strong distinguishing patterns for the less-represented class.

- is_sold_classifier_final.pkl — the final saved model (Gradient Boosting).
- is_sold_features_final.pkl — the list of final features used in training, including the engineered features.

9. Interactive Web App (Streamlit App)

The Web App Development team wrapped all of the project's outputs — insights and models — inside a single interactive web app built with the Streamlit library for Python, allowing any non-technical user to explore the project and interact with the models directly from a browser.

9.1 App Pages

#	Page	Function
1	Home	A welcome page showing a quick overview of the project, key figures (250 properties, 8 models), and a table of team members and their roles.
2	Price Prediction	An interactive form letting the user input property features (area, location, type, number of rooms, etc.) and get an instant price prediction using the saved regression model.
3	Sale-Status Classification (Is_Sold)	An interactive interface predicting the probability that a given property will be sold, based on its features, using the final classification model.
4	EDA Insights	Displays the five main charts and the key insights from the EDA stage interactively.
5	Models Comparison	A table and charts comparing the performance of all trained models across the regression and classification stages.

9.2 Interface Design and Technologies

- A fully custom design via a dedicated CSS stylesheet (style.css) and CSS code, including gradient-colored headers, specially designed buttons, and highlighted result boxes for displaying predictions.
- Multi-page app organization via the pages folder — the standard Streamlit approach for building multi-screen apps.
- Loading saved models (.pkl files) directly inside the app via the joblib library, to run predictions without needing to retrain.
- A custom logo and banner (assets/logo.png and assets/banner.png) to give the app a professional visual identity.

9.3 How to Run

Running the app requires installing the dependencies listed in requirements.txt (streamlit, pandas, numpy, matplotlib, seaborn, scikit-learn, xgboost, joblib, openpyxl), then running the following command inside the app folder: streamlit run app.py — making the app available locally via the browser.

9.4 Application Walkthrough (Screenshots)

The following screenshots were taken from the running application, showing each page and its main functionality.

Home Page

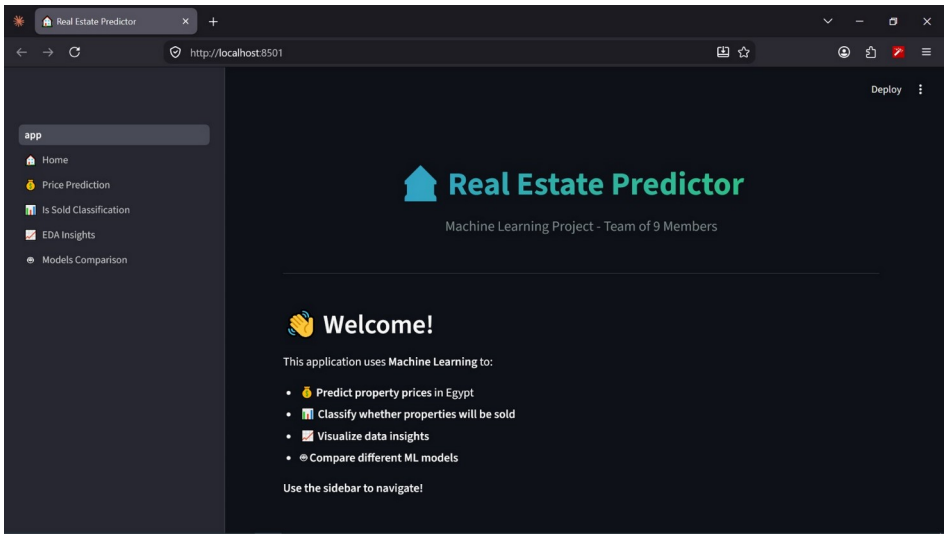


Figure 11: The landing page — a quick overview of the project, its purpose, and the four things it does (price prediction, sale classification, data visualization, and model comparison).

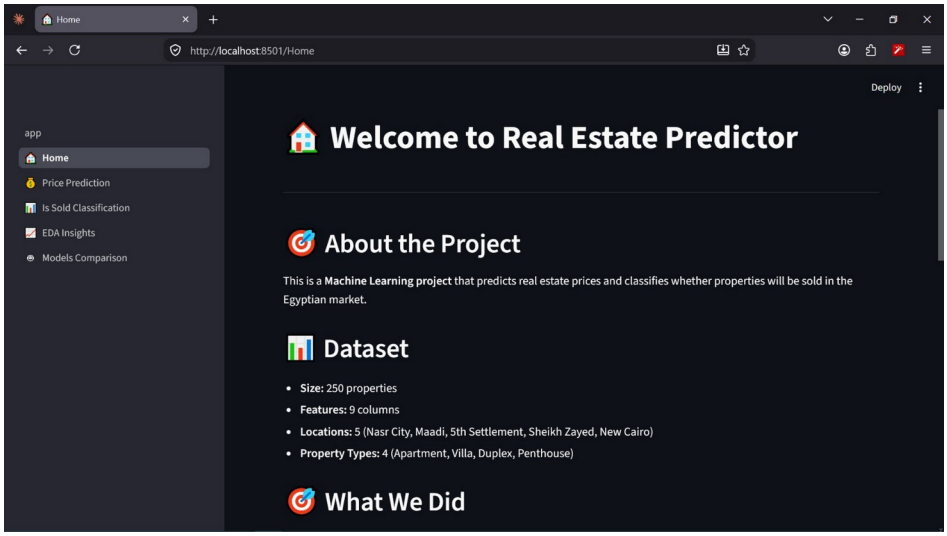


Figure 12: The dedicated Home page, showing the project description, dataset details (250 properties, 9 features, 5 locations, 4 property types), and a summary of the work carried out.

Price Prediction Page

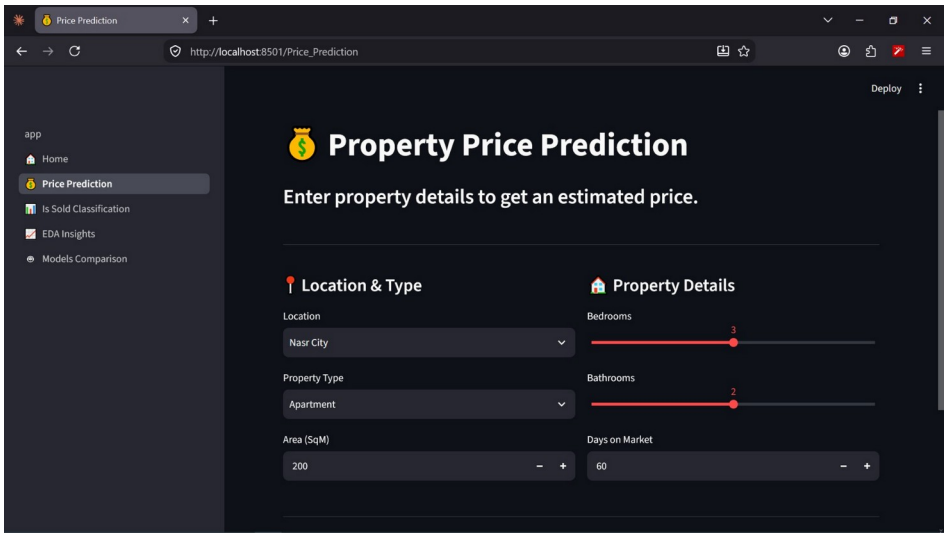


Figure 13: The Price Prediction page — the user selects the location and property type, and adjusts the area, number of bedrooms/bathrooms, and days on market to get an instant estimated price from the saved regression model.

Is Sold Classification Page

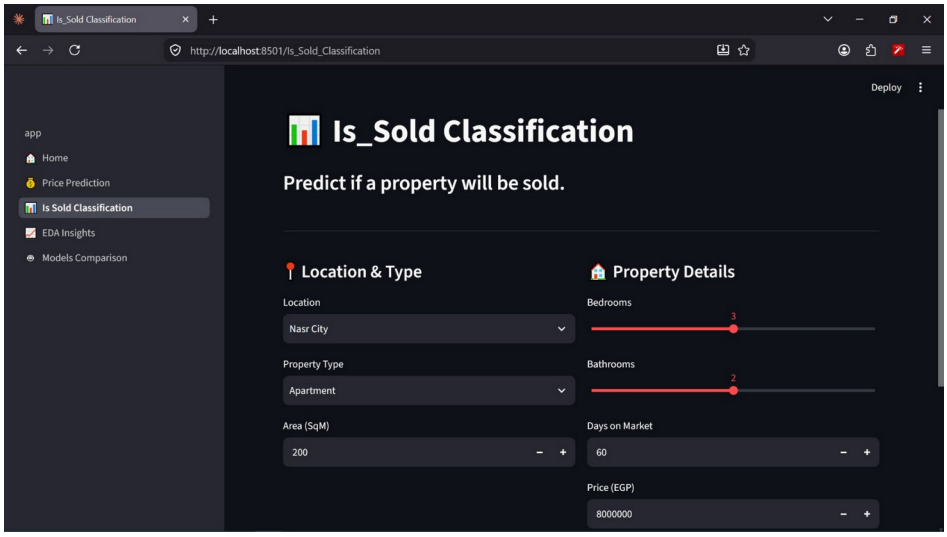


Figure 14: The Is_Sold Classification page — similar inputs to the price page, with an added price field, used to estimate the probability that the property will be sold, based on the final classification model.

EDA Insights Page

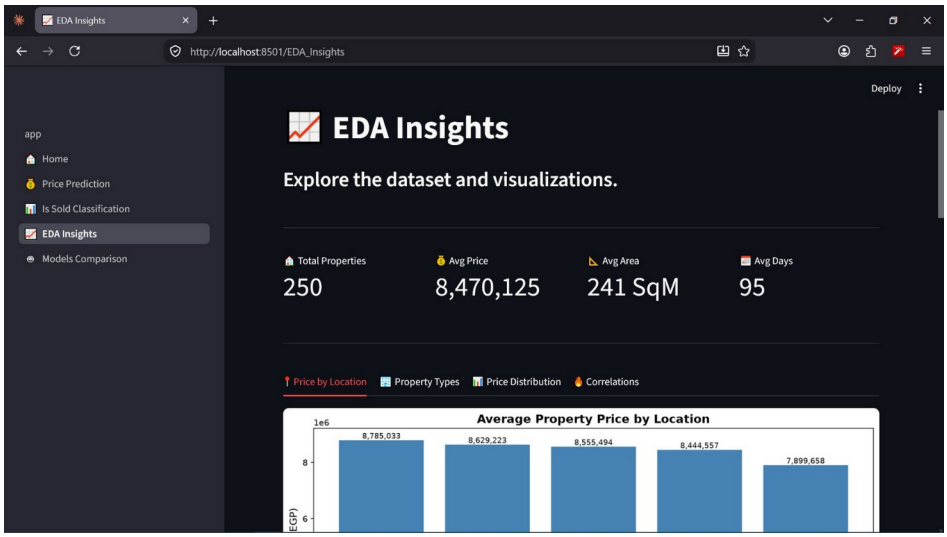


Figure 15: The EDA Insights page — key dataset statistics (250 properties, average price of EGP 8,470,125, average area of 241 sqm, average 95 days on market), with interactive tabs for the price-by-location, property-type, price-distribution, and correlation charts.

Models Comparison Page

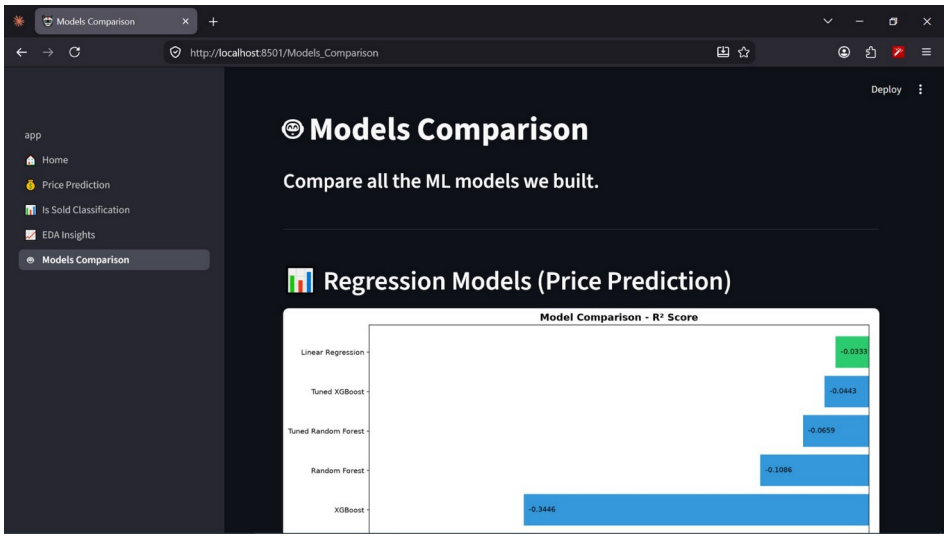


Figure 16: The Models Comparison page — a chart comparing the R^2 scores of all regression models, letting the user visually see which model performed closest to zero (best relative performance).

10. Tools and Technologies Used

The team relied entirely on Python as the primary language for the project, alongside a set of libraries and tools specialized for each stage. The table below summarizes the main tools used and their purposes:

Library / Tool	Purpose
Python	The primary programming language for the entire project.
Jupyter Notebook	The interactive development environment for writing code and documenting each analytical step sequentially.
numpy & pandas	Reading, processing, and cleaning data, and performing computational and

Library / Tool	Purpose
	statistical operations.
seaborn & matplotlib	Building explanatory charts (bar, pie, histogram, scatter plots, heatmaps).
scikit-learn	Data splitting, feature scaling (StandardScaler), building Linear Regression / Decision Tree / Random Forest / Gradient Boosting models, hyperparameter tuning (GridSearchCV), cross-validation (cross_val_score, StratifiedKFold), and computing evaluation metrics.
XGBoost	Building advanced regression and classification models based on Gradient Boosted Trees (XGBRegressor and XGBClassifier).
joblib	Saving trained models and preprocessing tools (Scaler), and reloading them later without retraining.
Streamlit	Building a multi-page interactive web app to display results and enable live predictions.
Custom CSS	Customizing the look and feel of the Streamlit app (headers, buttons, result boxes).
Microsoft Excel	Preparing the Final_EDA.xlsx file, which gathers the raw and clean data along with EDA summaries and correlations in one place for easy review.
Git	Version control system for organizing the ten-member team's work on the same project via well-organized, clearly named files and folders.

11. Task Distribution Among Team Members

The project was carried out by a ten-member team, working across four sub-teams (Data & Exploratory Analysis, Modeling, Web App Development, and Presentation), with clear links between each stage's outputs and the next stage's inputs. The team members are:

- Adam Yasser Sayed
- Mohammed Alaa korani
- Yehia abdellhmed Yehia
- Omar Mohamed Ahmed
- Mohamed Adel Elsayed
- Tony Milad farouq
- Mohamed Fareed Mohamed
- Peter mohep kerolse Gregs
- Mohamed Alaa El-Din Mahmoud
- Yousef Osama Awad

This distribution reflects a working methodology very similar to that of real data science teams in companies, where each sub-team specializes in part of the project pipeline, with clear handoff points between teams — such as the .csv and .pkl files that move from the Data team to the Modeling team, and then from the Modeling team to the Web App Development team.

12. Challenges and Limitations

Like any training project, the team faced a number of challenges and limitations that affected the quality of the final results, and it is important to document them transparently as part of the learning process:

12.1 Small Data Size

The dataset contains only 250 properties — a relatively small size for training robust and reliable machine learning models, especially after splitting it into training and test sets (about 200 for training and 50 for testing), which makes the results more sensitive to random fluctuation.

12.2 Weak Relationship Between Features and Price

As shown by the EDA stage and the modeling results, the available features (area, number of bedrooms, location, type) do not explain price variation well, which is likely why all regression models produced negative R^2 values. This is likely due to the semi-random or random nature of the pricing in this training exercise's data, and not necessarily a reflection of a real real-estate market.

12.3 Class Imbalance in the Classification Problem

An imbalance was observed between the number of sold and unsold properties in the sale-status prediction problem (Is_Sold), which negatively affected the models' ability to accurately identify the less-represented class (Sold), despite using balancing techniques such as class_weight and scale_pos_weight.

12.4 Absence of Important Real Estate Features

The data does not include features that could be highly influential on price and purchase decisions in reality, such as floor number, finishing condition, proximity to services and transportation, building age, or the presence of a garden/garage — which limits the ability of any model, however advanced, to achieve high accuracy.

13. Future Recommendations

Based on the challenges outlined above, the team proposes the following recommendations to improve project results in future iterations:

- Expand the collected data by at least a few hundred additional properties, to increase the models' ability to generalize.
- Add new features with greater impact on price, such as floor number, finishing condition, proximity to services and transportation, and building age.
- Experiment with additional data-balancing techniques for the classification problem, such as SMOTE, to generate synthetic samples for the less-represented class.
- Experiment with additional regression and classification models such as LightGBM, CatBoost, or simple neural networks, and compare them against current results.
- Use broader cross-validation techniques (Nested Cross-Validation) when performing hyperparameter tuning, to avoid data leakage during evaluation.
- Deploy the app on a cloud platform such as Streamlit Community Cloud, to make it available to users outside the local environment.
- Add automated unit tests inside the app to verify the correctness of the preprocessing and prediction pipelines.

14. Conclusion

This project represents a practical, end-to-end application of the data science lifecycle — starting from raw data and ending with a ready-to-use interactive web app, passing through cleaning, exploratory analysis, and the building, comparison, and tuning of multiple regression and classification models. The ten-member team succeeded in organizing their work clearly across four sub-teams, in a way that simulates a real data science team's working environment.

Although the regression models' quantitative results were modest (negative R^2 values, due to the weak relationship between the available features and price), and the classification results were moderate (62% accuracy, due to class imbalance and small data size), the project's real value lies in the methodology followed: learning how to clean and analyze data; trying and comparing several algorithms through an organized, scientific approach; understanding the reasons behind weak performance rather than hiding them; and finally turning these results into a tangible, interactive product. The recommendations above provide a clear roadmap for improving this work in future iterations.

15. Appendix: Project File Structure

The structure below shows the actual organization of the project's folders and files as delivered:

- 1_Data/ — raw_data.csv, cleaned_data.csv, data_dictionary.md
- 2_EDA/ — 01_data_cleaning.ipynb, 02_eda_analysis.ipynb, eda_insights.md, five charts (chart_1 to chart_5)
- 3_Modeling/01_Regression/ — three notebook files (Youssef, Adel, Fareed), saved models (best_model.pkl, scaler.pkl, feature_columns.pkl, model_info.pkl), CSV result files, and charts
- 3_Modeling/02_Classification/ — three notebook files (V1, V2, V3), saved models for each version, and confusion matrices
- 4_Streamlit_App/ — app.py, pages/ folder (five pages), assets/, style.css, requirements.txt, README.md
- 5_Presentation/ and 6_Demo/ — dedicated to the presentation and demo materials for presenting the project to the evaluation committee
- Real_Estate_Final_EDA.xlsx — a comprehensive Excel file gathering the data and analyses in one place

This organized, sequentially numbered structure (1 through 6) reflects the project pipeline from start to finish, and makes it easy for anyone new to understand the workflow by browsing the folders in order.